

Strata — Hướng dẫn BA

Dành cho Business Analyst

strata-kb v1.4.0 · 2026-09-19

1. Tài liệu này dành cho ai

Bạn viết yêu cầu. Ticket, user story, tiêu chí chấp nhận — và với những tính năng lớn thì có cả mission plan để chia nhỏ chúng ra.

Vấn đề của bạn không phải là viết. Vấn đề là **bám nguồn**: làm sao để khi ticket ghi "trường này dài 4 ký tự, chữ và số", khẳng định đó đến từ một tài liệu có thật, ở một phiên bản xác định, và vẫn kiểm chứng được vài tháng sau khi có người hỏi nó lấy ở đâu ra.

Strata cho bạn ba thứ để làm việc đó:

1. **Tìm kiếm trên tri thức đã publish**, để bạn tìm được đoạn cần thay vì phải nhớ.
2. **Trích dẫn ghim phiên bản**, để ticket ghi lại đúng những gì tài liệu nói vào ngày bạn viết.
3. **Cổng Definition of Ready**, để một ticket không thể đến tay lập trình viên với trích dẫn hổng hoặc thiếu.

1.1 Repo BA làm gì và không làm gì

Có làm	Không làm
Đọc tri thức đã publish trên hub	Ingest tài liệu
Trích dẫn tri thức đó, ghim theo commit của hub	Tóm tắt bất cứ thứ gì
Quản lý phiên bản ticket và mission trong Git	Publish tri thức
Kiểm soát chúng bằng CI	Đẩy lên hệ thống quản lý issue

Repository của bạn hoàn toàn **chỉ đọc** đối với cơ sở tri thức. Bạn trích dẫn, bạn không bao giờ đóng góp. Điều này là có chủ đích: nó giữ cho yêu cầu nghiệp vụ truy vết được tới một phiên bản tri thức mà không trao cho quy trình làm yêu cầu quyền thay đổi chính tri thức đó.

Trợ lý cũng không bao giờ tự mở ticket thay bạn. Nó tạo ra Markdown; bạn đọc, commit, và tự dán vào hệ thống quản lý issue.

2. Thiết lập một lần

```
pip install strata-kb
kb init --kind ba
```

2.1 Trở tới hub

```
# .kb/config.yaml
kind: ba
hub: https://github.com/acme/kb-hub.git
```

Đây là nơi `kb ticket lint` và `kb query` phân giải tham chiếu.

2.2 Nối trợ lý của bạn với hub

`.mcp.json` (Claude Code) và `.cursor/mcp.json` (Cursor) đã được scaffold sẵn, nối tới các công cụ tìm kiếm và trích dẫn của hub — nhưng qua hai placeholder, `${STRATA_KB_HUB_URL}` và `${STRATA_KB_HTTP_TOKEN}`, mà chưa ai điền giá trị.

Chạy `kb mcp-setup` (trong trợ lý: `/kb-mcp-setup`). Lệnh này hỏi URL HTTP cơ sở và token của hub — xin token từ người quản trị hub — ghi cả hai vào `.env`, đảm bảo git bỏ qua file đó, rồi xác minh chúng với hub, để một URL sai và một token bị từ chối trả về hai lỗi khác nhau:

Biến	Ví dụ
<code>STRATA_KB_HUB_URL</code>	<code>http://kb-hub.example.com:8321</code>
<code>STRATA_KB_HTTP_TOKEN</code>	bearer token của hub

Chạy lại `kb mcp-setup` một lần nữa (không kèm gi) sẽ đọc lại cả hai giá trị từ `.env` và xác minh lần nữa mà không cần nhập lại token. Nên dùng prompt ẩn hoặc biến môi trường `STRATA_KB_HTTP_TOKEN` thay vì cờ `--token` — cờ đó để lại token trong lịch sử shell của bạn.

`.mcp.json` và `.cursor/mcp.json` đọc hai biến đó từ môi trường tiến trình, không đọc trực tiếp từ `.env`, nên hãy nạp `.env` vào shell của bạn (`set -a; source .env; set +a`, hoặc dùng `direnv`) rồi khởi động lại trợ lý — MCP chỉ đọc môi trường lúc khởi động.

2.3 Mở repository trong trợ lý

`ba-ticket-author` và `ba-mission-plan` được dựng sẵn dưới dạng skill, command và prompt cho **Claude Code**, **GitHub Copilot** và **Cursor**. Bạn dùng cái nào thì pipeline cũng như nhau.

2.4 Cấu hình cổng CI

Thiết lập	Ở đâu	Là gì
<code>STRATA_KB_HUB</code>	Actions → Variables	Hub mà cổng dùng để phân giải tham chiếu
<code>KB_HUB_TOKEN</code>	Actions → Secrets	Token đọc — chỉ cần với hub riêng tư
<code>KB_FAIL_ON_STALE</code>	Variables , tùy chọn	Đặt giá trị bất kỳ để một sửa đổi phía trên làm cổng fail

Sau đó, trong branch protection, hãy **yêu cầu check** `kb-ticket-lint` trên nhánh mà ticket của bạn đổ về. Việc lint chạy trong CI tự nó không chặn merge; việc yêu cầu check mới chặn.

Pull request mở từ một fork không đọc được secret của repository, nên với hub riêng tư cổng sẽ fail ở đó kèm thông báo không kết nối được hub. Đó là cổng từ chối báo xanh khi chưa kiểm tra được, không phải lỗi mạng. Hãy merge đóng góp từ fork qua một nhánh trong chính repository này.

3. Viết một ticket

Gọi `/ba-ticket-author` và mô tả nhu cầu nghiệp vụ. Agent chạy một pipeline chín bước; phần việc của bạn là bước 1, 3 và 9.

1 Intake	bạn mô tả nhu cầu
2 Parent mission	tuỳ chọn – nêu mission mà story này thuộc về
3 Ground	kb_search; CHÍNH BẠN chọn section nào áp dụng
4 Draft	story, AC, use case, hai sơ đồ
5 Pin	kb_context_new những khối đã ghim
6 Lint	kb ticket lint cho tới khi DoR: PASS
7 Ground technical	SA điền Technical grounding; kb ticket check
8 Maturity review	hai lượt review độc lập, tối đa 3 vòng
9 Review → save	bạn đọc, commit, dán vào hệ thống issue

3.1 Intake

Hãy mô tả nhu cầu bằng ngôn ngữ nghiệp vụ: năng lực cần có, ai cần, và giá trị mang lại. Đừng vội viết giải pháp. Các bước sau của agent hoạt động tốt hơn khi xuất phát từ một vấn đề rõ ràng, thay vì từ một cách hiện thực đã chốt sẵn.

3.2 Bám nguồn — bước thực sự quan trọng

Agent gọi `kb_search` và trình cho bạn **mọi section ứng viên**, không chỉ cái nó thích nhất. Hãy đọc. Hãy chọn những cái thực sự áp dụng được.

Đây là bước duy nhất mà phán đoán nghiệp vụ của bạn không uỷ quyền được. Agent tìm được các đoạn ứng viên; nó không thể biết rằng §4.12 trên thực tế đã bị một chính sách nội bộ thay thế, hay hai section trông giống nhau lại khác nhau ở chỗ quan trọng trong tình huống này. Mọi thứ phía sau — bản nháp, việc ghim, phần hiện thực của lập trình viên — đều đứng trên lựa chọn này.

3.3 Soạn nháp

Agent điền vào template ticket: tóm tắt, user story, bối cảnh, tiêu chí chấp nhận, use case, và hai sơ đồ Mermaid (một sequence diagram và một business flow). Mọi khẳng định chạm tới một tiêu chuẩn đều mang trích dẫn `[doc-id §section]`.

3.4 Ghim

Khi bạn xác nhận các section, agent gọi `kb_context_new` và những khối `## KB context` trả về. Khối đó ghi lại commit của hub tại thời điểm này. Từ đây ticket không còn nói "xem sổ tay" — nó nói "đây chính xác là điều sổ tay nói vào ngày viết ticket này, và đây là cách kiểm chứng".

3.5 Lint

`kb ticket lint` chạy, agent sửa những gì nó báo, và lặp lại cho tới khi `DoR: PASS`.

3.6 Maturity review

Khi lint đã pass, agent chạy hai lượt review độc lập dựa trên `docs/review-rubric.md` — một lượt chấm **Business coverage**, một lượt chấm **Dev implementability**. Nó áp dụng sửa đổi rồi review lại, tối đa ba vòng hoặc tới khi cả hai trực đạt từ 4 trở lên.

Khoảng trống nào nó không tự khép được sẽ trở thành một câu hỏi mở có người phụ trách, viết là `OPEN(<owner>)`, thay vì một phỏng đoán. Kết quả nằm ở mục `## Review record` của ticket.

Hãy đọc các câu hỏi mở. Đó là chỗ agent nói cho bạn biết nó biết là nó chưa biết.

3.7 Bạn review

Bản nháp được lưu vào `tickets/<ticket-id>.md`. Bạn đọc, commit, rồi dán vào hệ thống quản lý issue. Trợ lý không bao giờ làm việc đó thay bạn.

3.8 SA ghim phần kỹ thuật

Bước 7 làm việc này thay bạn. Khi lint báo `DoR: PASS`, agent lưu bản nháp rồi gọi `/sa-ticket-ground` trên chính file đó như một subagent riêng. Lệnh này điền mục `## Technical grounding` do SA sở hữu — service, file, bảng, route, external, volumes, lệnh healthcheck và device reservations, lệnh test — mỗi dòng là một section id lấy từ tài liệu `<repo>-code` trên hub. Nó không bao giờ sửa mục của bạn; một phát biểu nghiệp vụ mâu thuẫn với sự thật trong code sẽ được trích lại ở đó, không bị sửa.

Code chưa tồn tại không phải là thiếu dữ liệu, mà là một quyết định thiết kế. SA đề xuất nó thành một dòng trong `## Technology decisions` của mission cha (trạng thái `OPEN`, chủ sở hữu là một con người) và ghi `[NEW: D<n>]` trở tới dòng đó. Chỉ những gì code đã có mà tài liệu không chứng minh được — luồng nội bộ, tình huống lỗi, thân request — mới được gác dưới `Open decisions` cho lập trình viên.

`kb ticket check tickets/<ticket-id>.md` là cổng: `PASS` chỉ khi mọi id phân giải được, mọi `[NEW: D<n>]` trở tới một dòng `DECIDED`, và `Open decisions` rỗng. Một ticket không có mission cha thì không có bảng để giữ dòng, nên SA viết văn bản tự do `[NEW: <reason>]` thay vào đó — gate chấp nhận nó như một ghi chú; một mission sẽ cho quyết định đó một chủ sở hữu. Một dòng `OPEN` làm fail cổng và nêu tên chủ sở hữu — đổi nó sang `DECIDED` là quyết định của bạn, không bao giờ của agent. Bản bàn giao mang theo khối `## Needs input` liệt kê đúng những dòng đó.

Chỉ chạy `/sa-ticket-ground` bằng tay khi cần ghim lại một ticket sau lúc `<repo>-code` đã thay đổi.

4. Mission plan — cho tính năng lớn

Một tính năng trải dài nhiều user story thì làm **mission plan** trước. Việc nhỏ thì đi thẳng tới ticket. Mission không bao giờ là bắt buộc.

```
/ba-mission-plan
```

Pipeline là **Intake** → **Ground** → **Draft** → **Split** → **Ground services** → **Pin** → **Lint** → **Maturity review** → **Review**, và lưu ra `missions/M-<slug>.md`.

Một mission mang theo:

- Sơ đồ **C4 Level 1** (System Context) và sơ đồ **C4 Level 2** (Container).
- **Phân định phạm vi** — cái gì nằm trong, cái gì rõ ràng nằm ngoài.
- **Backlog user story** có id suy ra từ id của mission.

```
kb mission lint missions/M-checkout-revamp.md
```

Cổng kiểm tra: cấu trúc bắt buộc, có đủ cả hai sơ đồ, backlog đúng dạng với id suy ra từ id mission, và mọi trích dẫn phân giải được ở phiên bản hub đã ghim. Cảnh báo `0/N US drafted` là bình thường — các ticket chưa tồn tại.

Cái vẫn thuộc **phán đoán của bạn**: backlog đã thực sự đầy đủ hay chưa.

4.1 Nối ticket với mission

Khi soạn một story từ backlog, hãy nêu mission cha. Agent sẽ:

- đọc `missions/<mission-id>.md` và lấy tiêu đề story từ dòng backlog,
- ghi `> Parent mission: <mission-id>` trên một dòng riêng ngay dưới tiêu đề ticket,
- lưu ticket thành `tickets/<mission-id>-US<n>.md`.

Chính tên file đó là thứ phép kiểm tra liên kết ngược tìm kiếm. Bỏ dòng parent đi vẫn hợp lệ — chỉ là ticket không truy vết được về mission của nó.

4.2 Đánh số

Backlog có lỗi hỏng số thứ tự là chuyện bình thường. Nếu bạn bỏ một story, hãy để số của nó nghỉ hưu. Đánh số lại sẽ làm hỏng tên file của những ticket đã soạn.

4.3 SA điền danh sách service

Bước 5 làm việc này thay bạn. Khi bạn đã xác nhận backlog và `## Sequencing` đã điền, agent gọi `/sa-ticket-ground --mission` và nó điền `## Services & order` — mỗi dòng một `svc.<name>` lấy từ tài liệu `<repo>-code` trên hub, cột `Depends on` chép từ chính record đó. Service mà mission sẽ tạo mới mang `[NEW: D<n>]`, trở tới một dòng SA thêm vào `## Technology decisions` để bạn quyết định. Gate sau đó báo FAIL đúng những dòng đó cho tới khi bạn chuyển chúng sang `DECIDED` — đó là kết quả mong đợi, không phải lỗi. Chỉ ở lớp năng lực: không tên file, không bảng, không route — những cái đó thuộc về `## Technical grounding` của từng ticket, điền sau ở mục 3.8.

Ở repo skeleton, mỗi dòng mà tài liệu architecture chứng minh được đều trích dẫn nó ngay trong ô Decision (`[<arch-doc> $<section>]`), và agent dừng ở bước 5b — **Decide** — để trình cả bảng cho bạn một lần. Bạn chuyển những dòng có trích dẫn mà bạn xác nhận sang `DECIDED`; dòng không có trích dẫn giữ `OPEN` kèm chủ sở hữu. Bạn quyết định ở đây, một lần, không phải từng ticket. Story đầu tiên của mission như vậy là **foundation slice**: những gì tài liệu architecture nói phải có trước mọi story tính năng, và mọi story khác phụ thuộc vào nó trong `## Sequencing`.

4.4 Ticket kế tiếp

```
kb mission next
```

Mọi story trong backlog của toàn bộ `missions/` được báo là `done` (id của nó nằm trong một dòng `hist.*` của `<repo>-svc` trên hub — thứ `kb svc note` ghi lúc lập trình viên bàn giao và CI publish khi merge), `drafted` (đã có file ticket), `ready` (chưa có ticket, mọi story trong `Depends on` đã done, mọi dòng D chặn nó đã `DECIDED`) hoặc

`blocked` (nêu rõ lý do). Báo cáo kết thúc bằng `Next: <us-id> - <title>; /ba-ticket-author` không kèm tham số sẽ chạy lệnh này trước và đề xuất đúng story đó. Một phụ thuộc mới chỉ `drafted` vẫn chặn: `done` nghĩa là đã merge. Thứ tự giữa các mission được ghi trong ô `Depends on` bằng story của mission khác (`M-<other>-US<n>`).

Lệnh chỉ đọc và thoát 0 sau khi in báo cáo. Hub chưa cấu hình hoặc không kết nối được, `-svc` chưa publish, hay không mission nào có dòng `Grounded on:` đều thành một dòng `note:`, không bao giờ là lỗi — không story nào được đánh `done` chừng nào hub chưa trả lời được.

5. Dùng tri thức về code

Ngoài tài liệu nghiệp vụ, hub còn giữ hai tài liệu cho mỗi repo sản phẩm, do chính workflow của repo đó publish: `<repo>-code` (cấu trúc do máy sinh — `svc.*`, `db.*`, `api.*`, `int.*`, `cmd.*`, `struct.tree`) và `<repo>-svc` (trách nhiệm do người biên soạn — mỗi service thực sự để làm gì).

Các skill `ba-ticket-author` và `ba-mission-plan` không tự đọc hai tài liệu này. Khi ticket hay mission cần tên service, bảng, route hay file, chúng viết `%%TODO: verify against codebase%%` kèm một câu hỏi mở có chủ, thay vì đoán — rồi `/sa-ticket-ground` (§3.8, §4.3) trả lời những câu đó từ `<repo>-code`, vào đúng mục do SA sở hữu — `## Technical grounding` ở ticket, `## Services & order` ở mission. Mọi id nó viết ra đều được `kb ticket check` đối chiếu với tài liệu; cái tài liệu không chứng minh được — luồng nội bộ, chế độ lỗi, nội dung request — được gác lại dưới `Open decisions` cho lập trình viên, người có code trong tay.

Chính tham chiếu section, viết đúng dạng một trích dẫn thật:

```
<repo>-code $svc.<name>
<repo>-svc $svc.<name>
```

Một lưu ý quan trọng. `<repo>-svc` dùng để bám sơ đồ. Nó không bao giờ thay thế được một trích dẫn nghiệp vụ trong tiêu chí chấp nhận. Một mã, định dạng, enum hay ngưỡng mã hoá một tiêu chuẩn thì vẫn phải đến từ một section nghiệp vụ đã ghim, không phải từ phần mô tả trách nhiệm của một service.

6. Cổng kiểm tra những gì — và không kiểm tra những gì

6.1 Lỗi (cổng fail)

- **Có đủ các mục bắt buộc:** Summary, User Story, Background, Acceptance Criteria, Use cases, cả hai sơ đồ Mermaid, KB context, Definition of Ready.
- **Tối đa 10 tiêu chí chấp nhận.** Nhiều hơn mười mục `- [ ]` dưới `## Acceptance Criteria` sẽ làm fail cổng. Hãy tách ticket — gộp hai điều kiện vào một AC cho vừa mức trần chính là điều luật này sinh ra để chặn.
- **Đúng một user story.** Nhiều hơn một dạng `As a ... I want ... so that ...` dưới `## User Story` sẽ làm fail cổng. Mỗi story một ticket.
- **Mọi tham chiếu trong `## KB context` phân giải được** tại commit hub đã ghim. Không có tham chiếu hổng hay sai định dạng.
- **Mọi trích dẫn `[doc-id $section]` trong nội dung đều có một tham chiếu đã ghim đứng sau.**

Trích dẫn viết theo dạng trần kiểu cũ vẫn được tính, kèm cảnh báo nhắc bạn đặt vào ngoặc vuông.

6.2 Cảnh báo (vẫn là phán đoán của bạn)

Cảnh báo	Vì sao không phải lỗi
Một tiêu chí chấp nhận chạm tới tiêu chuẩn nhưng không có trích dẫn	Công cụ không thể xác định đáng tin cậy tiêu chí nào chạm tới tiêu chuẩn
Chất lượng nghiệp vụ của story	Máy không kiểm được
<code>## Review record</code> thiếu, rỗng, hoặc còn nguyên chỗ trống mẫu	Lint thấy được mục đó, không thấy được review có thực chất hay không
Tham chiếu đã cũ	Section được trích đã bị sửa đổi phía trên. Vấn phân giải được nên không hỏng — nhưng bạn nên xem lại.
Một tham chiếu đã ghim mà nội dung không trích dẫn	Đó có thể là phần nền mà ticket không cần dẫn lại
Một tiêu chí chấp nhận ra lệnh một shell command (<code>docker compose ...</code> , <code>curl ...</code> , <code>grep ...</code> , hoặc hai lệnh nối bằng <code>&&</code>)	Công cụ chỉ gắn cờ hình dạng đó; AC có nên đổi sang mô tả kết quả quan sát được hay không là phán đoán của bạn — bản thân câu lệnh thuộc về <code>## Test data & verification</code> hoặc kế hoạch

6.3 Bắt công fail khi trích dẫn cũ

```
kb ticket lint tickets/ABC-123.md --fail-on-stale
kb mission lint missions/M-checkout.md --fail-on-stale
kb resolve tickets/ABC-123.md --status-only
```

Hoặc đặt biến repository `KB_FAIL_ON_STALE` để công CI làm điều đó cho mọi ticket.

6.4 Vì sao workflow không lọc theo đường dẫn

Workflow được dựng sẵn tên là `kb-ticket-lint` và chạy trên mọi pull request, không chỉ khi có thay đổi dưới `tickets/` hay `missions/`. Đó là chủ đích.

GitHub không bao giờ tự tạo ra trạng thái pass cho một job chưa từng khởi động. Một bộ lọc `paths:` trên một check *bắt buộc* sẽ khiến mọi pull request không đọng tới hai thư mục đó chờ mãi một check không bao giờ chạy. Thay vào đó job luôn khởi động, tự xem diff, và điều phối theo thư mục — thoát sạch kèm thông báo khi không có gì để lint.

Tên workflow cũng có trước công mission. Nó được giữ nguyên để tương thích với branch protection; đổi tên sẽ khiến những repository đã bật bảo vệ nhánh bị kẹt ở một check không còn tồn tại.

7. Khi một trích dẫn trở nên cũ

Tri thức thay đổi. Một ticket viết hồi tháng Ba dựa trên tài liệu bị sửa vào tháng Năm không được phép âm thầm mang nghĩa khác đi.

```
kb resolve tickets/ABC-123.md      # ok | stale | broken, theo từng tham chiếu
kb diff hr-handbook --against <rev> # thực sự đã đổi những gì
```

Trạng thái	Nghĩa là gì	Làm gì
ok	Nội dung đã ghim vẫn khớp với bản đang publish	Không phải làm gì
stale	Nguồn đã bị sửa đổi sau khi bạn trích dẫn	Chạy <code>kb diff</code> ; quyết định xem tiêu chí chấp nhận có cần cập nhật không
broken	Tham chiếu không còn phân giải được	Bám nguồn lại và ghim lại bằng <code>kb context new</code> hoặc <code>kb context_new</code>

`kb diff` cho biết chính xác cái gì đã dịch chuyển: tiêu đề section, dòng tóm tắt, phần tóm tắt, bản gốc đầy đủ, section được thêm và bị xoá, và thứ tự tài liệu có đổi hay không.

Phần lớn các trường hợp cũ là vô hại — một lỗi chính tả được sửa ở phía trên. Một số trường hợp là yêu cầu không còn khớp thực tế nữa. Công cụ chỉ cho bạn biết cần xem section nào; phán đoán vẫn là của bạn.

8. Làm việc không cần trợ lý

Mọi thứ agent làm, bạn đều làm được ở terminal.

```
kb query "thời hạn hoàn tiền" --tags payments
kb get payments-spec 4.2 --level 12
kb context new --refs "payments-spec $4.2, payments-spec $4.3"
kb ticket lint tickets/ABC-123.md
kb mission lint missions/M-checkout.md
```

`kb context new` in ra khối trích dẫn; dán nó vào ticket dưới mục `## KB context`.

9. Xử lý sự cố

Hiện tượng	Nguyên nhân	Cách xử lý
Lint: "ref does not resolve"	lỗi section sai, hoặc nội dung đã chưa được publish	Chạy lại <code>kb query</code> rồi ghim lại. Kiểm tra PR trên hub đã merge chưa.
Lint: "citation not backed by a pinned ref"	Nội dung trích dẫn thứ mà khối KB context không ghim	Thêm tham chiếu qua <code>kb context_new</code> , hoặc bỏ trích dẫn đó
Lint: "missing required heading"	Thiếu một mục bắt buộc — hoặc mục đó chỉ nằm bên trong khối code	Đưa tiêu đề thật ra ngoài khối code
Lint pass cục bộ, fail trên CI	Chưa đặt <code>STRATA_PB_HUB</code> trong Actions variables, hoặc hub nặng từ mã thiếu <code>KB_HUB_TOKEN</code>	Cấu hình cả hai; xem mục 2.4
Cổng fail trên PR từ fork	Fork không đọc được secret của repository	Merge qua một nhánh trong chính repository này
Troj ly không kết nối được hub	<code>STRATA_PB_HUB_URL</code> hoặc <code>STRATA_PB_HTTP_PORT</code> chưa đặt hoặc sai	Chạy lại <code>kb setuphub</code> (<code>kb setuphub --help</code> này chẵn đoạn chính xác cái nào sai; chạy lại không kèm gì sẽ xác minh lại mà không cần nhập lại token)
<code>kb setuphub</code> báo "token bị từ chối" dù đã an được token mới từ người quản trị hub	Chạy lại không kèm gì chỉ đọc lại đúng token cũ từ <code>setup</code> ra — lệnh chỉ hỏi lại khi tiến độ đã chưa có gì	Chạy <code>STRATA_PB_HTTP_PORT=token=token</code> <code>kb setuphub</code> , hoặc xoá dòng <code>STRATA_PB_HTTP_PORT</code> trong <code>.env</code> rồi chạy lại
<code>kb query</code> không tìm thấy gì	Tài liệu chưa được publish, hoặc tag quá hẹp	Bỏ <code>--tag</code> ; hỏi chủ hub xem PR publish đã merge chưa
Sau khi nâng cấp mọi thứ đều broken	Ticket mang khối ghim theo định dạng cũ	Ghim lại bằng <code>kb context_new</code>

10. Tóm tắt lệnh

Lệnh	Mục đích	Mã thoát
<code>kb mcp-setup [--hub-url URL] [--no-verify]</code>	Ghi thông tin xác thực HTTP MCP của hub vào <code>.env</code> và xác minh chúng	<code>0</code> ok, <code>1</code> thiếu giá trị hoặc kiểm tra kết nối thất bại
<code>kb query <text> [--tags t]</code>	Tìm trong tri thức đã publish	<code>0</code>
<code>kb get <doc> <section></code>	Lấy một section	<code>0</code>
<code>kb tags</code>	Liệt kê từ vựng tag đã publish	<code>0</code>
<code>kb context new --refs "..."</code>	Sinh khối trích dẫn đã ghim	<code>0</code> ok, <code>1</code> tham chiếu không phân giải được
<code>kb resolve <file></code>	Kiểm tra các tham chiếu của ticket	<code>0</code> tất cả ok, <code>1</code> có hỏng, <code>2</code> có cũ
<code>kb diff <doc> --against <rev></code>	Sửa đổi đã thay đổi những gì	<code>0</code>
<code>kb ticket lint <file></code>	Cổng Definition of Ready cho ticket	<code>0</code> PASS, <code>1</code> FAIL, <code>2</code> cũ
<code>kb mission lint <file></code>	Cổng Definition of Ready cho mission	<code>0</code> PASS, <code>1</code> FAIL, <code>2</code> cũ

Slash command: `/ba-ticket-author`, `/ba-mission-plan`.

11. Nâng cấp repository

```
kb init --kind ba
```

Lệnh này **chỉ định tới file dựng khung** — CI workflow, các file bọc skill, command và prompt, và template dưới `docs/` — và chỉ ghi đề khi nội dung khác với template mới.

`.kb/config.yaml`, `.kb/index.yaml` và `.claude/settings.json` không bao giờ bị `kb init` thường định dựng tới.

--force thay thế thẳng cả ba, không hợp nhất: `.kb/config.yaml` bạn đã tự sửa, bao gồm `kind`, `repo_id` và `asset_store`; `.kb/index.yaml`; và toàn bộ `.claude/settings.json` của bạn, bao gồm `hooks`, `permissions` và `model`. Hãy dùng một cách có chủ đích.

Nội dung của riêng bạn dưới `tickets/` và `missions/` không phải là file dựng khung. `kb init` không bao giờ đọc, ghi hay ghi đề bất cứ file nào bạn soạn ở đó — thứ duy nhất nó đặt vào hai thư mục này là một file `.gitkeep` rỗng, để Git theo dõi được thư mục trước khi bạn có ticket đầu tiên.